

Task 1:

Write a script that computes an estimate for the constant π . Here are the steps involved:

1. Prompt the user for a number of trials N . For best results make sure you select an N greater than one million.
2. Define a variable `n_circle` and set it to 0
3. Generate two (x, y) random numbers in range $[0,1]$. **Hint:** Here is a short code snippet that shows how you can generate these numbers:

```
import random
x = random.random()
y = random.random()
print(x, y)
```
4. Compute the distance between (x, y) and the origin of the circle $(0,0)$
5. Using this, determine if the distance less than or equal to 1. If yes then increment counter `n_circle` since the point falls inside the circle.
6. Repeat steps 3 to 5 N times

The estimate for π is then given by $4 * n_circle / N$

Task 2:

Suppose you've decided to ~~invest~~ gamble your money in Bitcoin. The price of Bitcoin is highly volatile, and changes randomly each day. Each day, the price of Bitcoin can go up by as much as 10%, or down by as much as 20% (so, an initial "investment" of \$100 can be worth up to \$110, or as little as \$80, after one day).

Use a while loop to figure out how long it will take before you've lost at least half of your money.

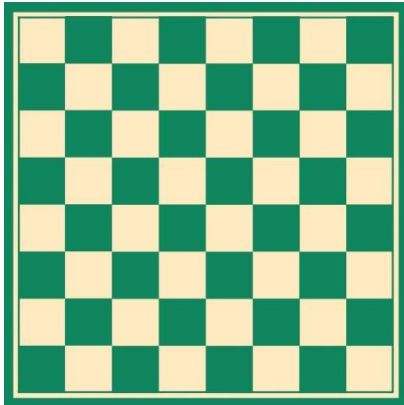
Hint: The following expression can be used to calculate the new value of your investment after each day, relative to its prior value:

```
(randrange(31) + 80) / 100
```

(for example, a value of .88 means your investment is worth 88% of what it was worth; a value of 1.06 means it's worth 106% of what it was (6% more than previous)). To make this work, you'll need to run **from random import randrange**. This is an example of importing a *function* from a *module* built in to Python. We'll learn more about both of those next week.

Task 3:

The game of chess is played on an eight-by-eight board, where cells alternate white and black, across both rows, and columns (strictly, white-and-black is not required – there are boards that are white-and-green, white-and-blue, etc). For example, here's a white-and-green board:



Here, we'll write some Python code that will generate something very much like a chess board – we'll print out an eight-by-eight grid, and put a `w` or `b` in each cell (or, if you prefer, `w` or `g`). The output might look something like this:

```
w b w b w b w b
b w b w b w b w
w b w b w b w b
b w b w b w b w
w b w b w b w b
b w b w b w b w
w b w b w b w b
b w b w b w b w
```

Note – this task is *tricky*! You'll need to pull together the material we talked about conditionals earlier, and the material on (doubly-nested) loops. You **can**, of course, just do this with a bunch of print statements, but that won't be quite so good practice.

To make the board come out right, you'll also (probably) want to tell `print` to not terminate every line automatically. You can do this as follows:

```
print("some text goes here", end="")
```

This will terminate each line of printed text with the empty string, instead of a customary newline.